

Lab 04

Getting Started with Remote Repos

Today's plan

01 Remote repos

Multiple ways to connect a repo to GitHub

02 Push troubleshooting

Reading errors, fixing them, avoiding force-push mistakes

03 Sync + GitHub features

README anatomy, browser editing

04 Reading Checkpoint #3

Reading

GitHub reminders

We know that GitHub is a platform that **hosts code**, providing **version control and collaboration features** and enabling to work with anyone in the world.

It has multiple other features like

- handling from small to very large projects (like [numpy](#)) ,
- tracking projects,
- reviewing and incorporating changes into your projects,
- even creating your own websites

Two paths to a remote repo

A. Start on GitHub

B. Start locally

A. Start on GitHub

1. Go to GitHub (log into your account)
2. Click on the + sign (top right corner) -> click on “New repository”
3. Enter the information:
 - a. Repository name - unique and descriptive
 - b. Description (optional)
 - c. Choose between Public or Private
 - d. Initialize repo with README (optional)
4. Click on Create Repository

You can now start adding files and making changes directly on GitHub.

A. Start on GitHub - and then?

Now that you have your **new** repo on GitHub, how do we start working on it?

We need a local copy of it (that's synced to it).

1. Copy URL (HTTPS) that shows up when you click on `<>` Code button
 - HTTPS and SSH are data transferring protocols
 - HTTPS requires less to be set up, but it's less secure
2. **git clone <link to repo>**
3. Work inside cloned folder

B. Start Locally

From previous labs, we know the steps.

You have created a repo locally (`git init`) and have committed some files

How to sync to it's remote version?

We also have to create a GitHub repo, same way as before. But now, we will make sure it's **empty**.

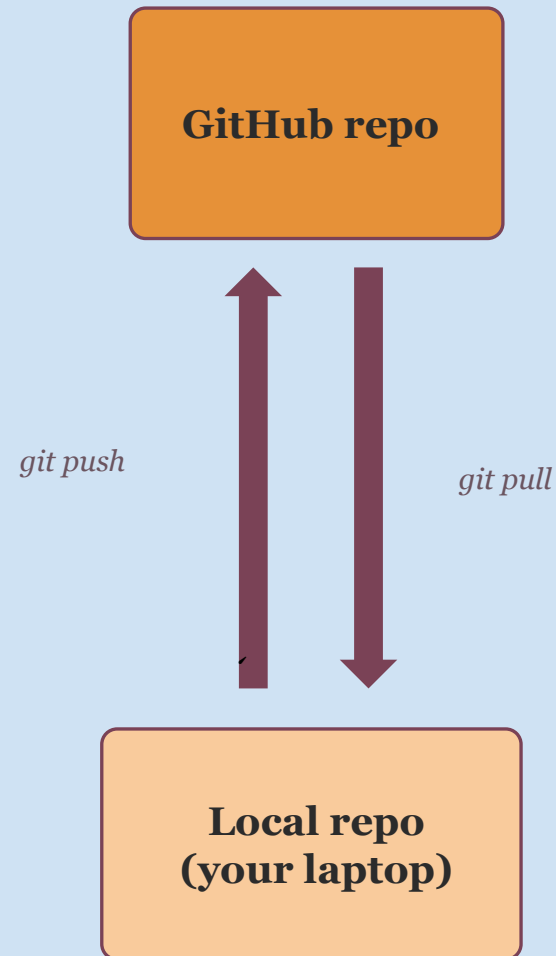
B. Start Locally - and then?

1. Copy URL (HTTPS) that shows up when you click on <> Code button.
2. But now go to your local repo and type:

```
git remote add origin <link to repo>
```

```
git push -u origin main (-u origin main only needs to be done once per branch)
```

A remote is just a named link



Locally, this connection is nicknamed 'origin'. It's a label you chose (or git chose for you), not a special word.

‘origin’

- **git remote -v**
 - shows every remote your repo knows about, and its URL
- You can name a remote anything — origin is just the git-assigned default
- A repo can have more than one remote at once

Now try for yourself - Lab notebook #4 - Part 1

"fatal: no upstream branch"

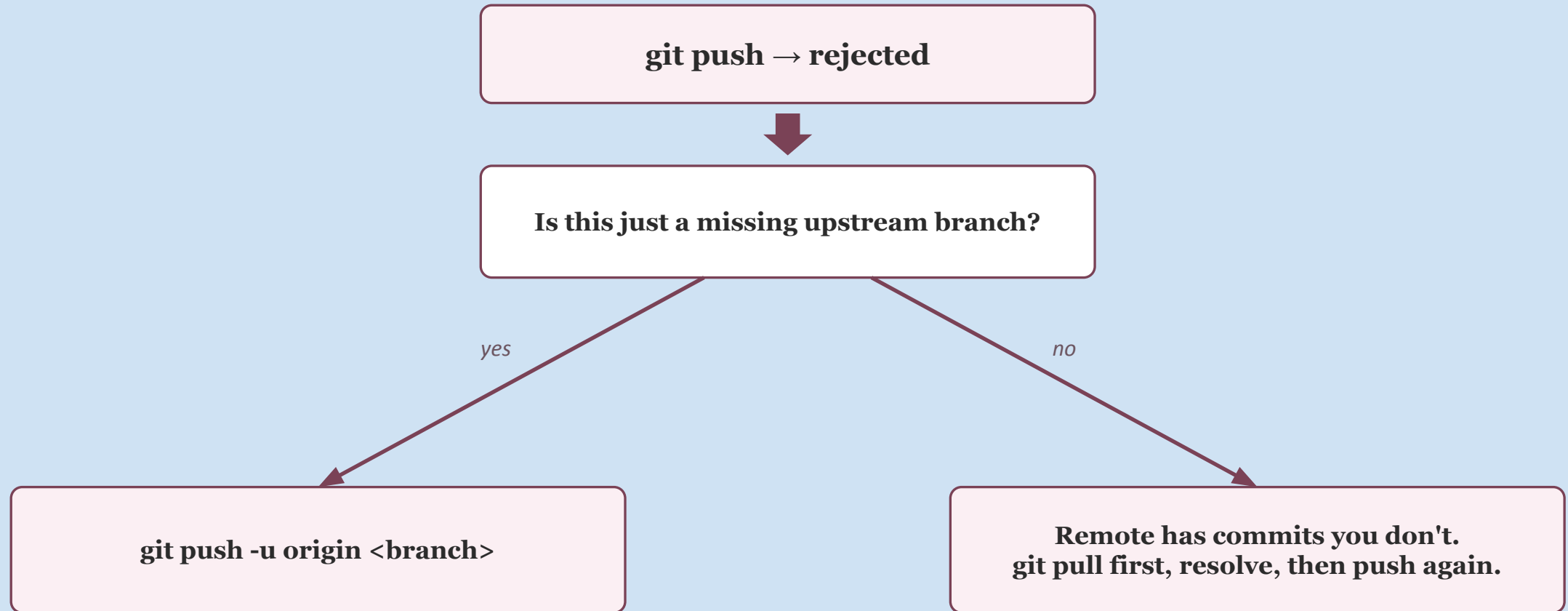
- Git doesn't know which remote branch your local branch should push to
- This is normal the first time you push a new branch

git push -u origin main

-u sets up tracking, so every push/pull after this just works with no flags

- After that first push, plain 'git push' and 'git pull' both work

Push errors



Force-push: know it, but mostly avoid it

- **git push --force**
 - overwrites the remote branch with your local history, no questions asked

The risk

- Anyone else's commits on that branch that you don't have locally are gone, permanently and for them too

Rule of thumb: never force-push a branch anyone else might be using. If in doubt, don't.

Reading your history

- **git log --oneline**
 - one line per commit: short hash + message
- **git log --oneline --graph**
 - the same thing, with a visual line showing how commits connect

Undoing (but the safe way)

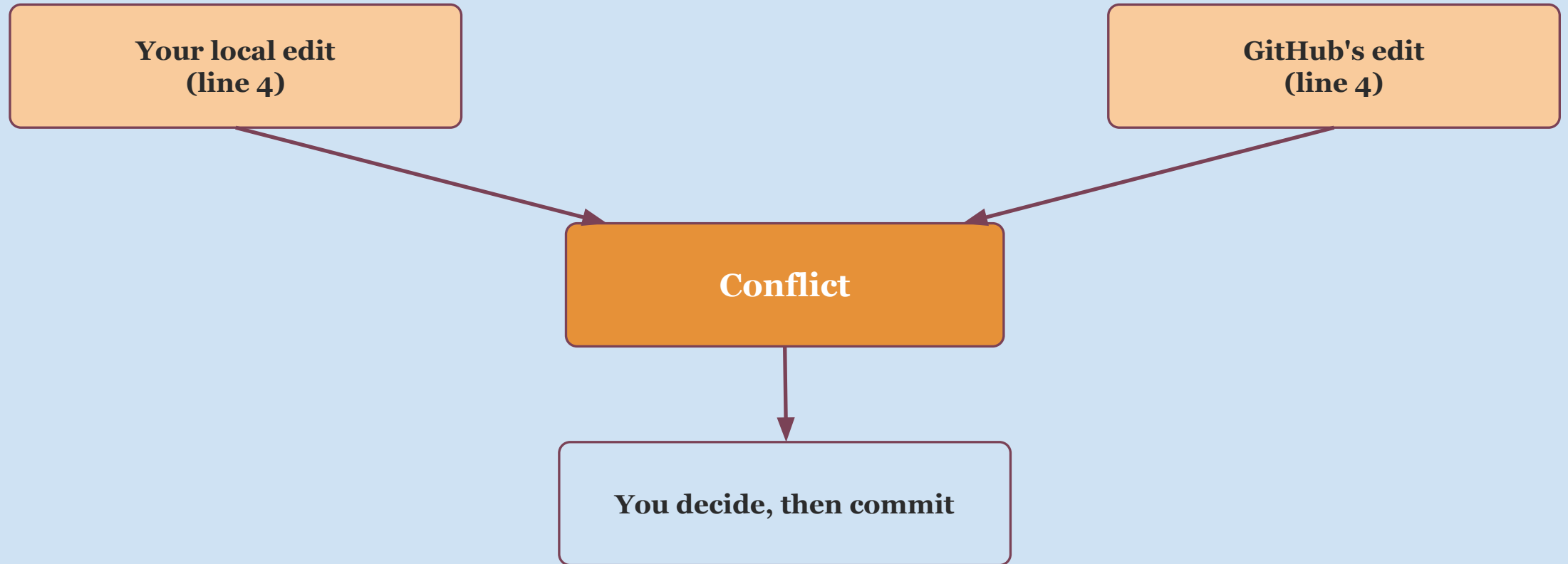
- **git restore <file>**
 - discard uncommitted local changes, back to the last commit
- **git revert <commit>**
 - undoes a commit by making a new commit that reverses it, the commit history will show it
- **git reset**
 - rewrites history, very powerful, don't default to it!

Note: if you've already pushed and others might have that history, prefer revert over reset.

Break

Now try for yourself - Lab notebook #4 - Part 2

When two edits collide



A rejected push means the remote moved. A conflict means it moved on the same lines you changed — git can't guess which version you want, so it stops and asks.

Reading conflict markers

```
<<<<<< HEAD  
your local version of the line  
=====  
the version from GitHub  
>>>>>> origin/main
```

- Everything between <<<<<< and ===== is yours (HEAD)
- Everything between ===== and >>>>>> is theirs (the incoming change)
- Edit the file by hand: keep what you want, delete the marker lines entirely
- **Then: git add <file> → git commit → git push (this commit completes the merge)**

A few repo settings worth knowing

In GitHub:

Go to Settings tab → visibility, rename, or delete, you can't do this by accident, GitHub asks for confirmation

If you renamed your

- GitHub redirects the old URL for a while, but update your local remote when you can:

```
git remote set-url origin <new-url>
```

- Deleting a repo requires typing the repo's name to confirm, there's no undo once you do

Anatomy of a good README

- Project title + one-line description of what it does
- Getting Started: install steps, then how to run it
- A short usage example
- License (and the badge GitHub renders from it)
- Markdown gives you real structure: headers, bold/italic, code blocks, links, tables — use it, don't write a wall of plain text

You have the option to edit on GitHub

- Pencil icon on any file → edit directly in the browser
- Commit dialog: write a message, choose where the commit goes
- On your own repo/fork, this commits straight to the branch you're viewing
- Commit history tab: every change, who made it, when

Good for: typo fixes, quick README edits. Not a replacement for real local development.

a README for you

GitHub has one special repo name: create a repo named exactly your username

- Check “Add a README” when creating it and it will render at the top of your GitHub profile page
- Repo has to be public

Some inspo:

- <https://github.com/jasonlong>
- <https://github.com/jvns>
- <https://github.com/RounakNeogy>
- <https://github.com/coderjojo/creative-profile-readme>

GitHub docs on profile READMEs:

<https://docs.github.com/en/account-and-profile/how-tos/profile-customization/managing-your-profile-readme>

Reading Checkpoint #4